

NYCU-EE IC LAB – Fall 2025

Lab03 Exercise

Design: Convex Hull

Data Preparation

1. Extract file from TA's directory:

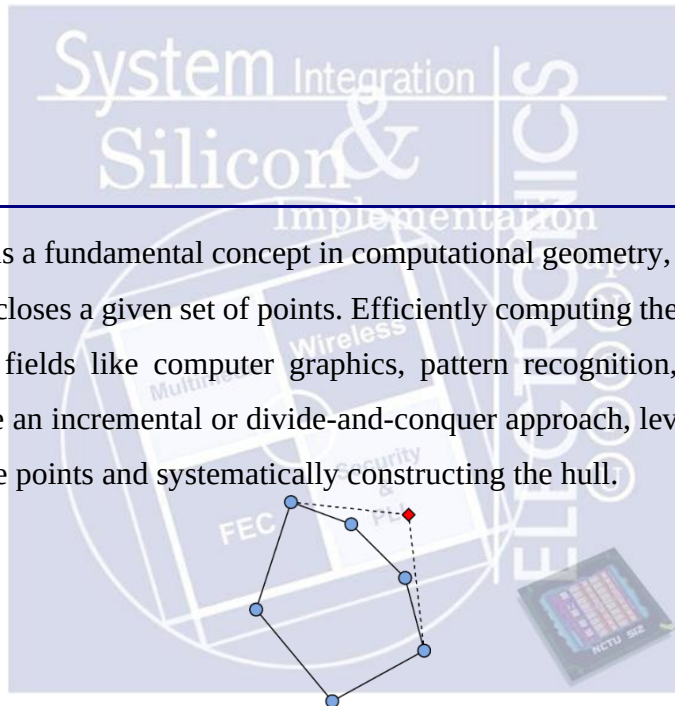
```
% tar xvf ~iclabTA01/Lab03.tar
```

2. The extracted LAB directory contains:

- a. **00_TESTBED**
- b. **01_RTL**
- c. **02_SYN**
- d. **03_GATE**
- e. **09_SUBMIT**

Design Description

The **Convex Hull** is a fundamental concept in computational geometry, representing the smallest convex polygon that encloses a given set of points. Efficiently computing the hull is crucial, making it a foundational tool in fields like computer graphics, pattern recognition, and robotics. Effective algorithms typically use an incremental or divide-and-conquer approach, leveraging a set's geometric properties by sorting the points and systematically constructing the hull.



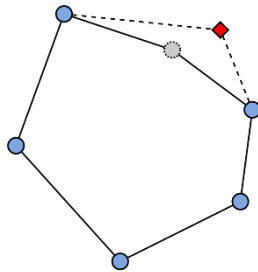
In this lab, your task is to solve the convex hull problem by creating a design and a verification pattern to validate it. Your design should process a series of incoming points, identify the vertices that form the convex hull, and discard any points that lie inside the resulting polygon. Your pattern, on the other hand, should integrate stimulus generation, environment checking, and response checking.

- **CONVEX.v**

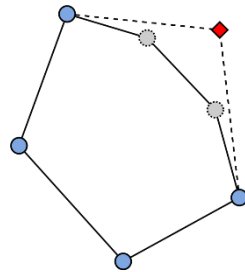
Whenever the Host system inputs a new point, **CONVEX** will update its data based on this point and calculate the new smallest convex polygon. Subsequently, **CONVEX** will only keep the vertices of the smallest convex polygon, discarding the remaining points and outputting the coordinates of these discarded points. The number of discarded points may vary depending on the situation; it could be a single point, multiple points, or no points at all. If the new point is identical to an existing vertex of the

polygon, it is directly discarded and output.

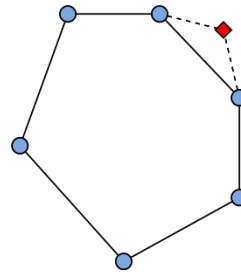
The following figures show operational examples, where the blue dots are the original polygon vertices, the red diamond is the newly added point, and the gray dots are the discarded points.



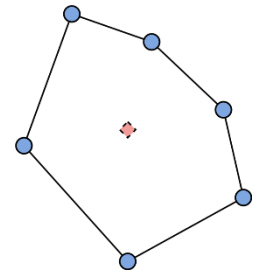
Discard one point



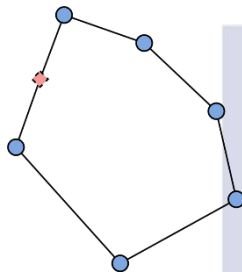
Discard multiple points



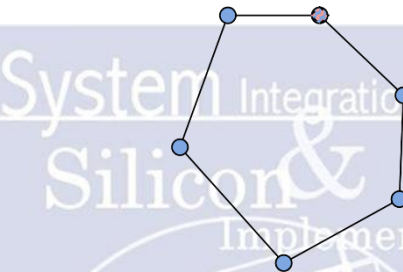
No output point



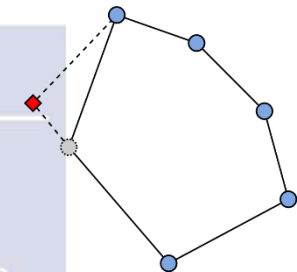
Added point discarded



Added collinear point discarded



Added coincident point discarded



Original polygon point becomes collinear and discarded

If the newly added point is outside the polygon, those vertices between the points of tangency should be discarded. On the other hand, the newly added point should be discarded whenever it is inside the polygon, on a vertex of the polygon, or on an edge of the polygon. The following figures demonstrate the process for first 6 points.

P0 (701, 434)

drop_num = 0

P1 (732, 403)

drop_num = 0

P2 (807, 734)

drop_num = 0

P3 (838, 72)

drop_num = 1

(x, y) = P1 (732, 403)

P4 (807, 322)

drop_num = 1

(x, y) = P4 (807, 322)

P5 (837, 696)

drop_num = 0

Notice that in the test patterns for this problem, the first 3 points will not be collinear and will certainly form a triangle. The convex polygons formed in the test patterns will not have more than 128 sides.

● PATTERN.v

In this lab, you need to read the patterns from the file `input.txt`. The format of `input.txt` is outlined below:

```
3           // 3 patterns in this file
438         // 438 points for pattern NO.1
701 434     // coordinate (701, 434) for point NO.1
732 403     // coordinate (732, 403) for point NO.2
...
409         // 409 points for pattern NO.2
376 208     // coordinate (376, 208) for point NO.1
268 221     // coordinate (268, 221) for point NO.2
...
403         // 403 points for pattern NO.3
345 666     // coordinate (345, 666) for point NO.1
283 764     // coordinate (283, 764) for point NO.2
...
```

1. The first number **N** at the beginning of the file is the number of patterns. The rest of the file is divided into **N** pattern groups.
2. The first number **P** at the beginning line of each group is the number of points in this pattern.
3. The rest of the lines in each pattern group contains an integer pair, each pair is a coordinate of input point that will be delivered to your design.

Please note that you should calculate the golden answer in the PATTERN. The golden answer from external files is NOT allowed.

All the above files are stored in 00_TESTBED.

Inputs and Outputs (CONVEX.v)

The following are the definitions of input signals

Input Signals	Bit Width	Definition
clk	1	Clock.
rst_n	1	Asynchronous active-low reset.
in_valid	1	High when all input is valid.
pt_num	9	The number of points for one pattern case, ranging from 4 to 500.
in_x	10	X coordinate of a new point. The value ranges from 0 to 1023.
in_y	10	Y coordinate of a new point. The value ranges from 0 to 1023.

The following are the definitions of output signals

Output Signals	Bit Width	Definition
out_valid	1	High when out is valid.
drop_num	7	The number of points discarded after a new point is added.
out_x	10	X coordinate of a discarded point.
out_y	10	Y coordinate of a discarded point.

4. All input signals will change at the clock negative edge.
5. All input signals are synchronized at the **negative edge** of the clock.
6. The **pt_num** will be given in the **first cycle of each pattern** when the **in_valid** is high.
7. For each pattern, **in_valid** will keep high for 1 cycle, along with **in_x**, **in_y**. After receiving the point, you need to process and output the answer right away. **The next in_valid, in_x, in_y will arrive in 1 negative edge of the clock after your out_valid is pulled down.**
8. The next input pattern will arrive between **1~4 negative edge of the clock** after the current **out_valid** is pulled down.
9. If multiple points are discarded, you can output the **out_x, out_y** coordinates **in any order**.

Specifications

1. Top module name: CONVEX (design file name: CONVEX.v)
2. It is asynchronous reset and active-low architecture. If you use synchronous reset (considering reset after clock starting) in your design, you may fail to reset signals.
3. The clock period must be less than **40 ns**.
4. The reset signal (**rst_n**) would be given only once at the beginning of simulation. All output signals should be reset. The pattern will check the output signal 100 ns after the reset signal is pulled low.
5. **The signals drop_num, out_x, out_y must be reset to 0 when the out_valid is low. And the out_x, out_y must be reset to 0 when drop_num is 0.**
6. **The out_valid should not be high when in_valid is high.**
7. **The latency for each input point is a maximum of 1000 cycles.** Latency is the time of the clock cycles between **the rising edge of the clk when the in_valid is high and the falling edge of the out_valid.**
8. The **drop_num, out_x, out_y** must be correct when **out_valid** is high.
9. If there are multiple points discarded, i.e. **drop_num > 1**, the **out_x, out_y** must be delivered for several cycles **continuously**, and **out_valid** should be high simultaneously.
10. The input delay is set to **0.5*(clock period)**.

11. The output delay is set to **0.5*(clock period)**, and the output loading is set to **0.05**.
12. The total cell area is limited in **2,000,000**.
13. The synthesized design must **not** contain any **latches**.
14. Gate-level simulation **cannot** include any **timing violations** without the notimingcheck command.
15. After synthesis, the slack at the end of the timing report should be **non-negative (MET)**.
16. Any words “error”, or “congratulation” cannot be used as variable name.

Grading Policy

1. **Function Validity:** 50% (Demo: Use your CONVEX.v and use the input.txt, PATTERN.v we provided)
2. **Pattern:** 30% (Demo: Use your PATTERN.v and use the input.txt, CONVEX_encrypted.v we provided.)
 - (15%) Pattern correctness. (Complete the simulation when design is correct.)
 - (2.5%) SPEC 4 (All outputs set to 0 when reset.)
 - (2.5%) SPEC 5 (Output value set to 0 when **out_valid** is 0.)
 - (2.5%) SPEC 6 (**out_valid** should not overlap with **in_valid**.)
 - (2.5%) SPEC 7 (Exceed max output latency.)
 - (2.5%) SPEC 8 (Answer should be correct.)
 - (2.5%) SPEC 9 (Multiple outputs should be in continuous cycles.)
3. **Performance:** 20 %
 - Area * total execution latency * cycle period
4. The grade for the 2nd demo would be **30% off**.
5. The design we provided will also check the pattern correctness. If the simulation cannot complete because of an error in your pattern, the pattern demo will fail.

Check the Result

Check Pattern

1. Include CONVEX_encrypted.v in your TESTBED.v.
2. Check each specification in terminal.

Specification	Command	Check the result on screen
Pattern correctness	<code>./01_run_vcs_rtl CORRECT</code>	Congratulations
SPEC 4	<code>./01_run_vcs_rtl SPEC_4_{1~4}</code>	SPEC-4 FAIL
SPEC 5	<code>./01_run_vcs_rtl SPEC_5_{1~5}</code>	SPEC-5 FAIL

SPEC 6	<code>./01_run_vcs_rtl SPEC_6_1</code>	SPEC-6 FAIL
SPEC 7	<code>./01_run_vcs_rtl SPEC_7_1</code>	SPEC-7 FAIL
SPEC 8	<code>./01_run_vcs_rtl SPEC_8_{1~10}</code>	SPEC-8 FAIL
SPEC 9	<code>./01_run_vcs_rtl SPEC_9_{1~2}</code>	SPEC-9 FAIL

- You can run `./07_check_pattern` to check all above specifications.
- Make sure the keyword printed on the screen is EXACTLY THE SAME as the one listed in the table above. You can print out other information according to your preference, but only ONE keyword from the table above is permissible and is indispensable.

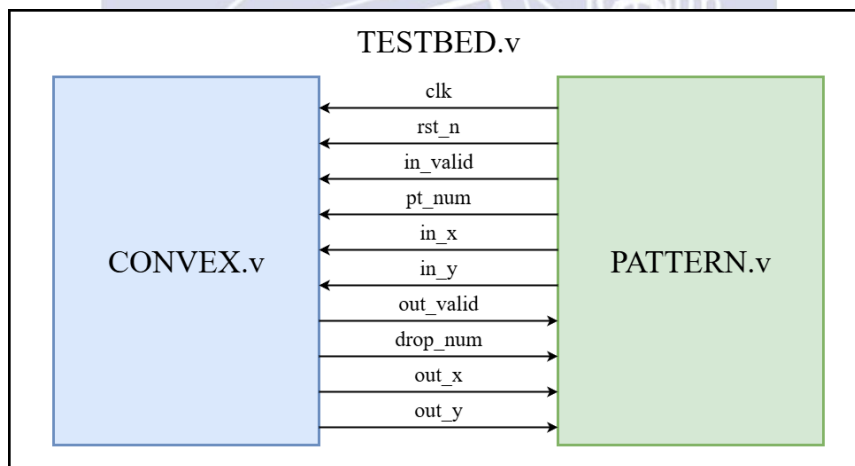
Check Design

1. Include `CONVEX.v` in your `TESTBED.v`.
2. Check each specification in terminal.

Specification	Command	Check the result on screen
Pattern correctness	<code>./01_run_vcs_rtl</code>	Congratulations

- You can run `./08_check_design` to check the design functionality.

Block Diagram



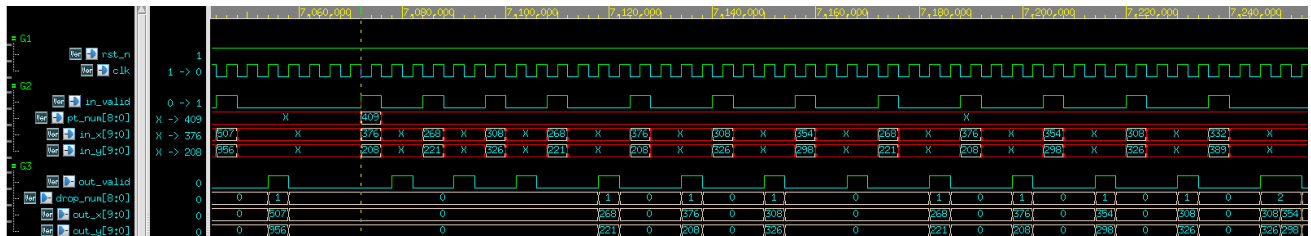
Note

1. Please submit `PATTERN.v`, `CONVEX.v` in **Lab03/EXERCISE/09_SUBMIT**
 - a. 1st_demo deadline: 2025/09/29(Mon.) 12:00:00
 - b. 2nd_demo deadline: 2025/10/01(Wed.) 12:00:00
2. **Please upload the following file under 09_SUBMIT:**
 - `PATTERN.v`, `CONVEX.v`
 - If your file **violates the naming rule**, you will **lose 5 points**.
 - Encrypting any uploaded files is prohibited and will result in a failed demo.

3. We provide the CONVEX_encrypted.v for you. You only need to complete the pattern and check the correctness of each design we provided.
4. You need to implement the CONVEX.v. We only check the functionality using our pattern.
5. There will be no hidden design, but **there will be hidden pattern.**

Sample Waveform

Inputs and Outputs



1. The waveform above is an example of the inputs and the outputs.
2. The next round of pattern inputs will come in 1~4 negative edge of the clock.
3. Each valid input point will arrive in 1 negative edge of the clock.
4. The outputs should come out continuously if multiple points are discarded.

